

20

26

PROGRAMAÇÃO PARA JOGOS I

Aula 01 — Apresentação da disciplina

2026.2 · quartas e sextas, 20h–22h · Prof. George · UFC

Quem sou eu



Profs. George e Alysson

george@virtual.ufc.br

alysson@virtual.ufc.br



Formação e trajetória

Onde estudei, por onde passei e o que me trouxe até jogos digitais.



Jogos que já desenvolvi

Projetos, protótipos e experiências com desenvolvimento de jogos.



Nesta disciplina

O que me motiva a ensinar programação de jogos — e o que espero de vocês.

Quem são vocês?

Uma resposta curta de cada um — isso calibra o ritmo do semestre inteiro.



Seu nome

E o curso / semestre em que está.



**Sua experiência
com programação**

Nunca programei? Já fiz uns scripts?
Trabalho com isso?



**Um jogo que
você ama**

Qualquer um — do Pong ao Elden
Ring. Sem julgamentos.

O que esta disciplina é — e o que não é



O que ela É

- Fundamentos de como jogos funcionam por dentro: game loop, colisão, padrões de projeto
- Um projeto construído de forma incremental e colaborativa, do zero
- Muita discussão, especificação e análise crítica — em sala



O que ela NÃO é

- Curso de Unity, Unreal ou Godot — engines aparecem, mas não são o foco
- Curso de arte, game design ou modelagem 3D
- A promessa de sair fazendo um jogo AAA — vamos entender por quê já já

Por que jogos são difíceis de programar?



Tempo real

O jogo não espera: a cada ~16 ms tudo precisa estar recalculado e desenhado.



Performance

Otimização não é luxo — é requisito. FPS baixo destrói a experiência.



Memória

Cenários, sprites, sons: tudo disputa um espaço que sempre acaba.



Hardware

Do celular modesto ao PC gamer caro — e o jogo tem que rodar bem em todos.

Por isso, começamos pequeno: **Pong** — simples o bastante para caber numa aula, rico o bastante para carregar o semestre.

Ementa da disciplina

- 1 Introdução e conceitos de jogos digitais
- 2 Estrutura geral de um jogo
- 3 Representação de personagens, cenários e objetos
- 4 Detecção de colisão
- 5 **Padrões de projeto: Game Loop, Update, Componentes, Observer, Singleton**
- 6 Motores de jogos
- 7 Desenvolvimento de protótipos

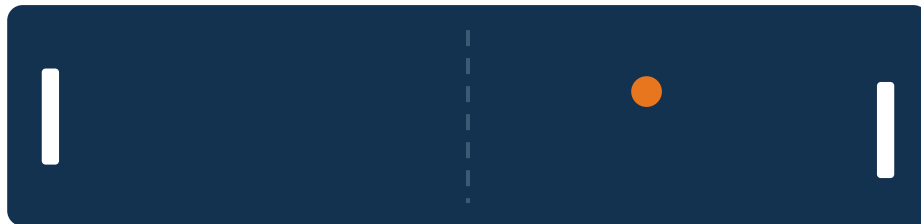


O coração do curso

Os padrões de projeto do livro *Game Programming Patterns* guiam nossa jornada — um de cada vez, sempre aplicados no nosso jogo.

Um jogo, um semestre: Pong como fio condutor

O mesmo jogo evolui a disciplina inteira. Cada conceito novo refatora o que já construímos — nada se perde, tudo se transforma.



simples de escrever · perfeito para refatorar

Cronograma do semestre — 2026.2

AGO

Fundamentos

Conceitos · SGDD · Game Loop · Colisão

SET

Padrões — parte 1

Game Loop avançado · Update · OO · Componentes

OUT

Padrões — parte 2

Observer · Singleton · Particionamento · Geometria×Textura · entrega do SGDD

NOV

Projeto + prova

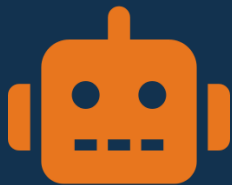
Práticas em grupo · orientações · avaliação escrita (25/11)

DEZ

Reta final

Game jam · apresentações do trabalho final (09 e 11/12)

Feriados no caminho: 16/10, 28/10 e 20/11 · cronograma completo aula a aula será publicado no canal da turma



Vamos falar de IA — sem tabus

A IA escreve código. Ela vai escrever código para vocês.
Eu mesmo usei IA para preparar este material — e vou dizer isso sempre que acontecer.

Fingir que ela não existe seria desonesto. A pergunta desta disciplina não é **SE** usar IA — é **COMO** usar bem.

O que a IA não faz por você



Especificar requisitos

Dizer o que o jogo deve fazer — com precisão e sem ambiguidade.



Entender mecânicas

Saber por que uma mecânica diverte e outra frustra.



Justificar arquitetura

Defender por que Observer aqui, e não um Singleton.



Criticar código ruim

Reconhecer quando a resposta da IA tem bug, gambiarra ou exagero.



Explicar suas decisões

Sustentar oralmente as escolhas que você fez — ou aceitou.



Trabalhar em equipe

Discutir, negociar e construir junto — em sala, ao vivo.

E é exatamente isso que será avaliado neste semestre.

Toda atividade terá uma cor



IA liberada

Desenvolvimento no dia a dia,
trabalho final, estudo.

*Use como quiser — mas o resultado é
responsabilidade sua.*



IA + análise crítica

Atividades em que usar IA é
obrigatório — e criticá-la também.

*Ex.: gerar código com IA e apresentar o
que ela acertou e errou.*



Sem IA

Discussões, arquitetura no quadro,
checkpoints orais, prova.

*Aqui o cérebro é só o seu. É onde a nota
mora.*

A cor de cada atividade será sempre anunciada com antecedência.

“

**Você pode entregar código
que a IA escreveu.**

**Você não pode entregar código
que você não consegue explicar.**

A regra de ouro da disciplina — todo checkpoint tem defesa oral.

GitHub: o diário de bordo do seu jogo

Repositório individual desde a Aula 04. Os fundamentos (commit, branch, pull request) serão explicados de forma conceitual — a burocracia dos comandos, o Copilot automatiza por você.



Histórico = evidência

Quem constrói aos poucos tem história. Projeto colado na véspera, não.



Quer aprender na mão?

Tutorial opcional: GitHub Skills (skills.github.com) — trilhas interativas e gratuitas.

Sua primeira missão: GitHub Education

A verificação leva até 72 horas — por isso é hoje, não semana que vem.

1

Crie sua conta no GitHub

github.com — se ainda não tiver. Use um nome de usuário apresentável: ele vai para o seu portfólio.

2

Aplique ao Student Developer Pack

education.github.com/pack — use seu e-mail institucional da UFC ou comprovante de matrícula.

3

Aguarde a verificação

Até 72 h. Vocês receberão e-mail de confirmação do GitHub.

4

Copilot gratuito ativado

Plano GitHub Copilot Student, sem cartão de crédito — pronto antes da Aula 04, quando criaremos os repositórios.



education.
github.com

gratuito para estudantes
verificados

Avaliação

N1

Avaliação escrita

- Prova teórica individual sobre todo o conteúdo
- Data: 25/11 (correção comentada em 27/11)
- Modalidade vermelha: sem IA

N2

Trabalho final

- Um jogo original, em grupo de até 3
- SGDD (30%) = 20% documento + 10% práticas de definição
- Jogo (70%) = 60% jogo + defesa oral + 10% práticas de desenvolvimento
- Apresentações: 09 e 11/12

As práticas em sala valem, somadas, 20% da N2 — quem falta, sente. Detalhes no próximo slide.

O trabalho final



Grupos de até 3

Jogo próprio, aplicando os conceitos e padrões vistos na disciplina.



SGDD primeiro (30%)

20% documento (entrega 23/10) + 10% práticas de definição: ideia, entidades/componentes e observadores do seu jogo.



Jogo funcional (70%)

60% jogo + defesa oral + 10% práticas de desenvolvimento. O histórico de commits avalia o processo incremental.



O que não vale

Projeto pronto da internet ou colado inteiro de uma IA. O histórico de commits denuncia — e a defesa oral, mais ainda.

Construção colaborativa: as funcionalidades do seu jogo nascem das discussões em sala — especificação, arquitetura e crítica.

Checkpoints e práticas avaliadas

DEFINIÇÃO → 10% do SGDD

19/08

Ideia do jogo

grupos formados + ideia apresentada

30/09

Entidades e componentes

as peças do seu jogo, esboçadas e defendidas

07/10

Observadores do jogo

os eventos do seu jogo, após exercitar no Pong

DESENVOLVIMENTO → 10% do jogo

09/09

Checkpoint 1

Pong rodando + defesa oral

25/09

Checkpoint 2

Pong com componentes, commitado

30/10

Arquitetura no quadro

diagrama consolidado, sem computador

04/11

Crítica de código IA

a nota está na sua análise

Cada prática compõe a nota do artefato que ela alimenta — definição constrói o SGDD, desenvolvimento constrói o jogo.

As ferramentas do semestre



Python + Pygame

Nossa linguagem e biblioteca de jogos



PyCharm

IDE oficial da disciplina — com o plugin do Copilot



GitHub Copilot

IA no editor, gratuita via GitHub Education



GitHub

Repositórios e histórico do projeto



Discord

Canal da turma — link será divulgado em breve

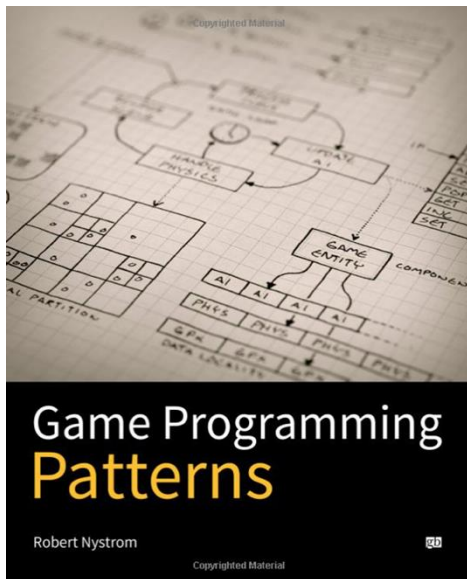


Sigaa

Notas, frequência e avaliações oficiais

Nada para instalar hoje — a instalação será guiada, juntos, na Aula 04 (21/08).

Referências



Game Programming Patterns

Robert Nystrom — o livro-guia dos nossos padrões de projeto. **Gratuito online:** gameprogrammingpatterns.com



Game Programming Algorithms and Techniques

Sanjay Madhav — leitura complementar, com abordagem independente de plataforma.

+ documentação do Pygame (pygame.org/docs) e GitHub Skills (skills.github.com)

É isso. Bora fazer jogos.

Próxima aula — sexta, 14/08: **O que é um jogo?**

conceitos · diversão e frustração · teoria do Flow

Não esqueça: **aplicar ao GitHub Education ainda hoje** → education.github.com

Dúvidas? george@virtual.ufc.br